$ mqtt sub factory/+/status --interval 2s

설비 통신 장애 대응 지원 웹앱

MQTT 로그·매뉴얼 기반 로컬 LLM 활용



정보통신공학 **22313584**

발표자 **류승범**

// 왜 이 주제인가

장애 대응은 아직 사람의 경험에 기대고 있다

2026.09 국립순천대 ICC / ㈜이음아이씨티

스마트팜 유지보수 사례

설비 장애의 원인 판단과 대응이 작업자 경험에 의존한다. 숙련 인력이 없는 현장에서는 빠르게 대응하기 어렵다.

2026.01 CES 2026 / 두산밥캣

Service.AI

정비 기술자가 장비별 매뉴얼과 정비 이력을 빠르게 확인하도록 돕는다. 수리 시간과 장비 정지 시간을 줄이는 것이 목표다.

→ 장애 대응 지식을 통신 로그와 문서에서 바로 확인할 수 있게 하자

출처: 국립순천대학교 보도자료(2026.09.14) / Bobcat Company, Doosan Bobcat Introduces Smarter Machines for Simpler Operation at CES 2026(2026.01.05)

// 문제

흩어진 로그와 자료, 남지 않는 점검 기록

01 **자료를 따로따로 찾아야 한다**

장애가 나면 통신 로그, 점검 매뉴얼, 과거 조치 기록을 각각 열어 봐야 한다.

02 **무엇을 점검했는지 남지 않는다**

점검 과정과 최종 조치가 기록되지 않으면 다음 대응에 쓸 수 없다.

목표: 한 화면에서 끝나는 대응 흐름

1. MQTT 메시지 수신
2. 규칙 기반 장애 판정
3. 장애와 로그 기록 (SQLite)
4. 근거 문서 기반 점검 안내
5. 사용자 점검과 조치 기록

// 설계 원칙

판정은 규칙이 하고, 설명은 AI가 돕는다.

RULE ENGINE

장애 유형, 발생 시각, 복구 여부는 프로그램이 정한다.

LOCAL LLM

매뉴얼과 로그를 바탕으로 가능한 원인과 점검 순서를 설명한다. 원인을 확정하거나 장치를 제어하지 않는다.

NO EVIDENCE

관련 매뉴얼이 없으면 모델을 호출하지 않고 근거 부족으로 표시한다.

// 구현 범위

PC 실습 환경으로 범위를 좁혔다

2s	10s	3s	1대	3종
상태 메시지 발행 주기	연결 이상 판정 (미수신)	응답 지연 판정 (요청 후)	모의 장치 RPI-01	통신 장애 유형
factory/RPI-01/status	CONNECTION_TIMEOUT	RESPONSE_DELAY	Python 시뮬레이터	중단 / 지연 / 형식 오류

기준의 성격

실제 생산설비의 운영 기준이 아니라 기능 검증을 위한 실습 규격이다.

범위 밖

Raspberry Pi 실기기 연동과 실제 산업 설비 검증은 향후 과제로 둔다. 매뉴얼과 조치 사례는 실습용으로 직접 작성했다.

// 시스템 구조

메시지는 왼쪽에서 오른쪽으로, AI는 필요할 때만

PC 시뮬레이터
RPI-01
Python 장치

→

Mosquitto
MQTT Broker
:1883

→

MQTT Worker
규칙 엔진
0.25s 판정 루프

→

SQLite
로그 / 장애
조치 / heartbeat

→

FastAPI
/api
조회 / 조치 / health

→

Vue 3
대시보드
장애 상세

status / request / response

Worker와 FastAPI는 **별도 프로세스**로 돌고 같은 SQLite 파일을 쓴다. 수신과 판정이 API 서버 상태에 묶이지 않는다.

AI 경로: 장애 상세에서 GET /api/events/{id}/ai-guidance

① 매뉴얼 선택
장애 유형별
manuals/*.md

→

② 절 검색
+ 연결된
통신 로그

→

③ Ollama
EXAONE 3.5
2.4B (로컬)

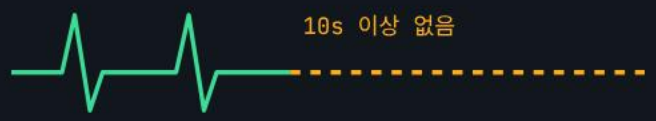
→

④ 안내+출처
제목 / 절
파일 / 줄

// 장애 판정 3종

세 가지 장애를 서로 다른 신호로 잡는다

CONNECTION_TIMEOUT

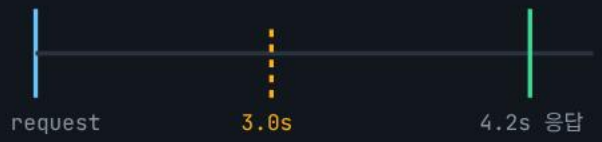


상태 메시지 수신 중단

마지막 유효 상태 메시지 이후 10초 동안 다음 메시지가 없으면 판정

재현: --mode disconnect

RESPONSE_DELAY



요청 대비 응답 지연

서버 요청 후 3초 안에 응답이 오지 않으면 판정. 늦게라도 응답이 오면 복구로 기록

재현: --mode delay (4.2초 후 응답)

INVALID_MESSAGE

```
{"device_id":"RPI-01","seq":42,...}
{"status": "ok"
```

필드 누락 / JSON 파싱 실패

메시지 형식 오류

필수 필드(장치 ID, 상태값, 순번, 타임스탬프)가 없거나 JSON이 깨지면 판정

재현: --mode invalid

// 오판 방지

감시 시스템이 멈춘 것을 장치 고장으로 보지 않는다

상황	Worker	Broker	장치 상태	장애 기록
Worker heartbeat가 3초 넘게 갱신 안 됨	stopped	unknown	unknown	하지 않음
Worker는 정상, Broker 연결이 끊김	running	disconnected	unknown	하지 않음
Broker 연결 상태에서 유효 메시지 10초 이상 없음	running	connected	timeout	CONNECTION_TIMEOUT

/api/health는 API 처리 상태와 Worker, Broker, 장치 heartbeat를 따로 보고한다. 자동 테스트로 두 경우를 검증했다.

// SQLite

통신이 복구된 것과 조치를 마친 것은 다른 일이다

devices

장치의 현재 상태와 마지막 수신 시각

communication_logs

수신한 MQTT 메시지와 검증 결과

fault_events

장애 유형, 발생/감지/복구 시각

action_records

체크리스트 선택과 사용자 조치 내용

	점검 기록 완료	점검 기록 필요
통신 정상화	**종료** 정상 처리된 장애	**기록 누락** 통신은 돌아왔지만 점검 내용이 없음
통신 끊김	**조치 후 대기** 점검은 했지만 메시지가 아직 안 옴	**대응 필요** 장애 진행 중

// RAG + LOCAL LLM

답변에는 항상 어느 문서의 어느 절인지가 붙는다

01

매뉴얼 선택

규칙 엔진이 정한 장애 유형으로 해당 매뉴얼을 고른다

02

절 검색

판정과 범위, 가능한 원인, 점검 순서 등 관련 절을 뽑는다

03

로그와 함께 생성

문서 발췌와 실제 통신 로그를 Ollama 모델에 넘긴다

04

출처 표시

답변 아래 문서 제목, 절, 파일, 줄 위치를 보여준다

프롬프트 제약

매뉴얼과 로그만 근거로 쓴다. 장애 원인을 확정하거나 판정을 바꾸지 않는다. 근거가 없는 부분은 "확인할 수 없음"이라고 쓴다.

출처 예시 (통신 중단)

title	MQTT 상태 메시지 미수신 점검
section	점검 순서
file	manuals/connection_timeout.md
model	exaone3.5:2.4b / timeout 60s

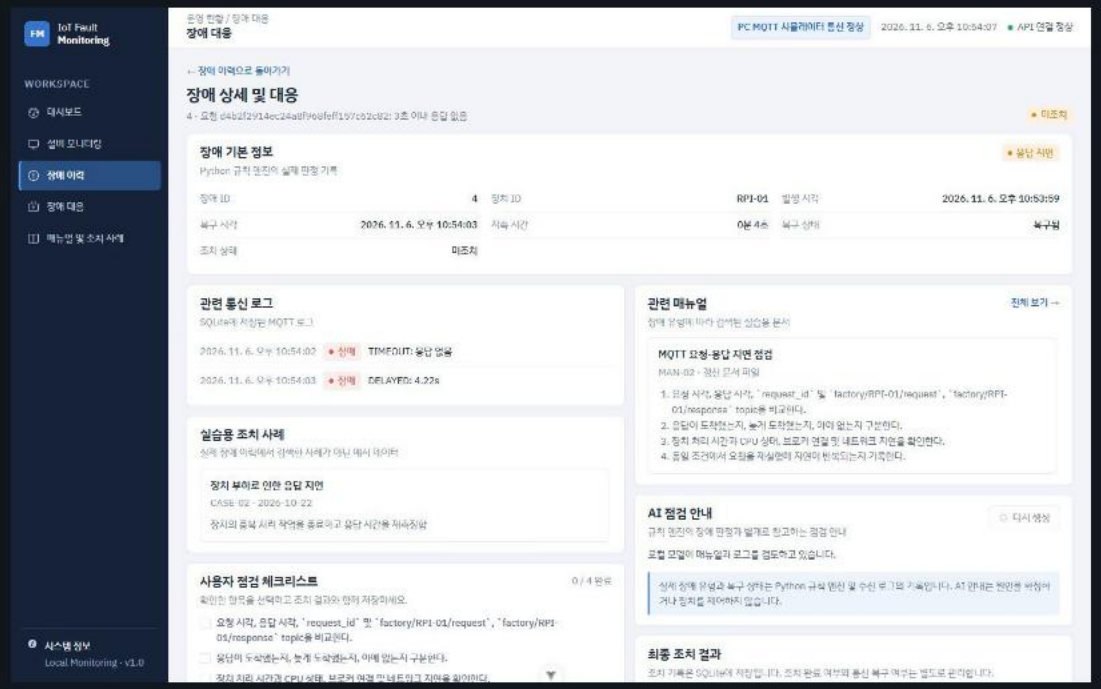
// Vue 3 대시보드

대시보드에서 상세까지, 대응에 필요한 정보를 한 곳에



DASHBOARD

장치 수신 상태, 장애 유형별 건수, 복구와 조치 현황



EVENT DETAIL

관련 로그, 매뉴얼, 조치 사례, AI 점검 안내, 체크리스트

// 대표 시나리오: CONNECTION_TIMEOUT

LIVE DEMO_

STEP 1

정상 수신 확인

--mode normal / 2s 주기

STEP 2

발행 중단

Ctrl+C / 10s 대기

STEP 3

자동 감지

장애 이력에 새 이벤트

STEP 4

로그, 매뉴얼, AI 안내

출처 위치 확인

STEP 5

체크리스트와 조치 저장

조치 완료 상태

STEP 6

정상 모드로 복구

복구 시각 기록

실시간 연결에 문제가 생기면 같은 과정을 녹화한 영상으로 대체한다.

// 장애 유형별 5회, 총 15회 (PC 격리 환경)

탐지는 판정 기준에서 0.27초 안쪽으로 일어났다

CONNECTION_TIMEOUT

10.106s

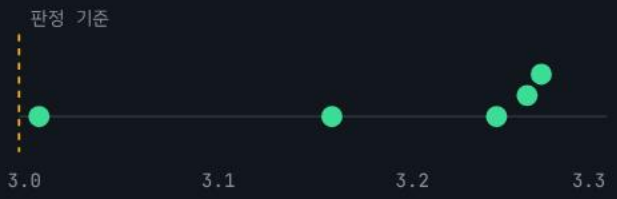
평균 / 범위 10.003 ~ 10.207s



RESPONSE_DELAY

3.190s

평균 / 범위 3.010 ~ 3.269s



INVALID_MESSAGE

5 / 5 판정

수신과 판정이 같은 시각 해상도로 기록되어 소요 시간은 **측정 불가**로 남겼다. 0초라고 주장하지 않는다.

Worker는 0.25초마다 판정한다. 측정 시작점은 통신 중단이 마지막 유효 상태 수신, 응답 지연이 요청 등록이라 두 값을 직접 비교하지 않는다.

// Ollama exaone3.5:2.4b, 5회 호출

첫 호출을 빼면 3.6초 안팎에 답한다



5 / 5
응답 생성 성공

4.664s
평균 (중앙값 3.638s)

8.908s
첫 호출 (모델 적재로 추정)

5회 모두 같은 형식 오류 이벤트(ID 77)에 대한 호출이다. HTTP 요청 시작부터 응답 종료까지의 시간이며, 다른 장애 유형으로 일반화하지 않는다.

// 응답 지연 이벤트, 실제 응답 1건을 매뉴얼·로그와 대조

답이 나왔다고 근거에 충실한 것은 아니다: 부분 적합

확인 항목	판정	근거
장애 유형과 문서 일치	적합	RESPONSE_DELAY에 response_delay.md의 네 절이 검색됨
매뉴얼에 있는 원인 후보	대체로 적합	처리 지연, CPU 부하, 재연결, 네트워크 지연, topic/request_id 불일치를 가능성으로 표현
로그에 없는 시각 단정	개선	프롬프트 수정 후 "마지막 정상 응답 시각" 단정이 사라짐
매뉴얼 밖의 원인과 점검	부적합	"소프트웨어 버그", "소프트웨어 업데이트 확인", "다른 네트워크에서 재시도"는 문서와 로그에 없음

출처 표시가 문장별 정확성을 보증하지는 않는다. 그래서 AI 안내를 확정 진단이나 자동 조치로 쓰지 않는다.

// 예외 처리 / 한계

AI가 멈춰도 감지는 멈추지 않는다

검증한 예외 처리

- **Ollama 응답 없음**: AI 안내만 사용 불가로 표시, 수신과 판정, 조회는 유지
- **관련 매뉴얼 없음**: 모델을 호출하지 않고 근거 부족으로 응답
- **Broker 끊김**: 장치 상태를 unknown으로 두고 고장으로 단정하지 않음
- **자동 테스트 43개** 통과 (판정, 중복 등록, 복구, 조치 분리, RAG)

한계

- 장치 1대, PC 격리 환경. 실제 네트워크 장애의 지연 분포는 대표하지 못함
- 탐지 실험 15회, AI 응답 5회로 표본이 작음
- AI 답변 5건의 본문을 저장하지 않아 근거 대조는 별도 1건만 수행
- 매뉴얼과 조치 사례는 직접 작성한 실습용 자료

// 정리

감지부터 조치 기록까지 하나의 흐름으로 이었다

한 것

- MQTT 통신 장애 3종을 **규칙으로 판정**하고 이력으로 남김
- 감시 시스템 장애와 장치 장애를 **구분**
- 통신 복구와 사용자 조치 완료를 **분리 관리**하고, 저장한 조치를 같은 유형의 다음 장애에서 **다시 보여줌**
- 매뉴얼 근거와 출처가 붙는 **로컬 AI 점검 안내**

다음 단계

- Raspberry Pi 5 실기기로 실제 Ethernet 단절 실험
- AI 답변 본문 저장과 문장별 근거 대조, 근거 밖 문장 차단
- 더 긴 운영 기간과 실제 매뉴얼, 조치 사례 축적

감사합니다

질문 받겠습니다



정보통신공학 **22313584**

발표자 **류승범**